

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a653a277aa8c35a46.jsonl`
- SHA-256: `e0dbc57eeb9524499bce1c72bafd88f706a3e81e7db6da8eccd2b14fab6ff96f`
- Source modified: `2026-06-10T16:31:20+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:27:42.327Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Savings oracle ignores available opening/closing bookends when the balance walk passes ? missing leading/trailing transactions persist as 'reconciled'

SEVERITY CLAIMED: high

FILE: src/muneem/parsers/savings.py (line 78-84, 194-205)

DESCRIPTION: reconcile_oracle returns True as soon as the per-row balance walk passes, never checking the printed opening/closing even when parse_statement_summary found them. The walk only verifies CONSECUTIVE rows that are present, so any omission at the statement's edges is invisible: a dropped first/last transaction line (the exact clusterer failure class seen in Axis May-2025 ? it was only quarantined because the dropped line was interior), a leading/trailing extract_tables table that failed to map (parse_savings silently keeps only oracle-endorsed tables, then re-checks the combined set whose walk still passes), or a truncated multi-page PDF. Empirically confirmed: with opening=10000.0 available, the oracle endorsed (a) a ledger whose entire leading half was missing (implied opening 9000 != 10000) and (b) a first row whose debit was misread as 9999 instead of 500 ? the first balance-bearing row's flows are never compared against anything. The HDFC parser has the same blind spot via its fallback: when parse_summary returns None (e.g. the final page carrying the SUMMARY block is missing ? the very scenario that also truncates transactions), the verdict downgrades to reconcile(stmt).ok alone and grants RECONCILED. Note validate via verify() correctly withholds trust without bookends (test_verify_unavailable_figures...), but neither the savings path nor HDFC's fallback uses that discipline.

EVIDENCE QUOTED: savings.py:78-81: `def check(rows): if reconcile\_running\_balance([(r.debit, r.credit, r.balance) for r in rows]).ok: return True` ? short-circuits before the opening/closing check. hdfc.py:449-453: `ok = (validate\_against\_summary(stmt, summary).ok if summary is not None else reconcile(stmt).ok)`. Empirical run: oracle(opening=10000, closing=8500) endorsed rows starting at balance 8000 with a 1000 debit (implied opening 9000) -> True; endorsed first-row debit=9999 against balance 9500 -> True.

SUGGESTED FIX: In reconcile_oracle, when opening is available require the implied opening (rows[0].balance + debit0 - credit0 == opening) and when closing is available require rows[-1].balance == closing, in ADDITION to the walk (i.e. reuse verify()'s bookend checks). Treat walk-pass-without-any-bookend as a distinct lower-confidence verdict rather than 'reconciled'. For HDFC, treat an absent SUMMARY as UNRECONCILED (verify() already does this) instead of falling back to the walk alone.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

```
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/,
```

testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).

- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service. Structured output only.

2. user (2026-06-10T16:27:48.000Z)

```
1      """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3      Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4      and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6      1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7      coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8      2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9      3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10     vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12     A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13     on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14     cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15     can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.
16
17     No statement content is logged; only counts/booleans travel out.
18     """
19
20     from __future__ import annotations
21
22     import re
23     from dataclasses import dataclass, field
24     from pathlib import Path
25
26     import pdfplumber
27
28     from .clustering import cluster_to_matrix
29     from .concepts import ConceptRow, StatementConcepts
30     from .engine import Profile, map_table
31     from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33     _MONEY_TOKEN = re.compile(r"^\d{2}(\d{2})?(\d{2})?", re.IGNORECASE)
34     _OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?\d{2})?", re.IGNORECASE)
35     _CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?\d{2})?", re.IGNORECASE)
36
37
38     @dataclass
39     class SavingsParse:
40         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42         account_number: str
```

```

43     rows: list[ConceptRow] = field(default_factory=list)
44     verified: bool = False
45
46
47     def parse_statement_summary(text: str) -> StatementConcepts:
48         """Read opening/closing balance from statement text (for the total-reconcile
oracle).
49
50         Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
51         """
52         opening = _OPENING_RE.search(text)
53         closing = _CLOSING_RE.search(text)
54         return StatementConcepts(
55             opening_balance=parse_amount(opening.group(1)) if opening else None,
56             closing_balance=parse_amount(closing.group(1)) if closing else None,
57         )
58
59
60     def txn_line_predicate(profile: Profile):
61         """A line is transaction-like if it carries one of the profile's dates *and* a money
token.
62
63         Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
64         address blocks that would otherwise wreck column inference).
65         """
66         patterns = profile.date_patterns
67
68         def keep(texts: list[str]) -> bool:
69             has_date = any(p.match(t) for p in patterns) for t in texts)
70             return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
71
72         return keep
73
74
75     def reconcile_oracle(opening: float | None, closing: float | None):
76         """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""
77
78         def check(rows: list[ConceptRow]) -> bool:
79             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
80                 return True
81             if rows and opening is not None and closing is not None:
82                 return reconcile_total(rows, opening, closing)
83             return False
84
85         return check
86
87
88     def _reconstruct_by_balance_delta(
89         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
90     ) -> list[ConceptRow]:
91         """Recover debit/credit from the running balance when columns won't split cleanly.
92
93         Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
94         it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
95         single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
96         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
97         a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance

```

```

98         to fix the first row's direction; returns ``[]`` if it can't form a clean series.
99         ""
100         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
101         rows = [(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
102         if not rows or opening is None:
103             return []
104         ncols = max(len(r) for r in rows)
105
106         def is_date(s: str) -> bool:
107             return any(p.match(s) for p in patterns)
108
109         def money(s: str) -> float | None:
110             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
111
112         # date column = the leftmost of the (first few) columns with the most date cells
113         date_counts: dict[int, int] = {}
114         for r in rows:
115             for c in range(min(ncols, 3)):
116                 if c < len(r) and is_date(r[c]):
117                     date_counts[c] = date_counts.get(c, 0) + 1
118         if not date_counts:
119             return []
120         top = max(date_counts.values())
121         date_col = min(c for c, n in date_counts.items() if n == top)
122
123         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
124         if len(txn_rows) < 2:
125             return []
126
127         # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
128         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
129         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
130         money_cols = []
131         for c in range(ncols):
132             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
133             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
134             if m >= 1 and m > d:
135                 money_cols.append(c)
136         if len(money_cols) < 2:
137             return []
138         balance_col = max(money_cols)
139         flow_cols = [c for c in money_cols if c != balance_col]
140
141         text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
142         narr_col = (
143             max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
144             if text_cols
145             else None
146         )
147
148         out: list[ConceptRow] = []
149         prev = opening
150         for r in txn_rows:
151             bal = money(r[balance_col]) if balance_col < len(r) else None
152             if bal is None:
153                 return [] # a running balance on every row is required for the delta
154             amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
155             debit = credit = None
156             if amt is not None:
157                 if bal < prev - tol:
158                     debit = amt

```

```

159         elif bal > prev + tol:
160             credit = amt
161         else:
162             debit = amt # zero net with an amount present ? reconciliation will
flag it
163         out.append(
164             ConceptRow(
165                 txn_date=r[date_col],
166                 narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
167                 debit=debit,
168                 credit=credit,
169                 balance=bal,
170             )
171         )
172         prev = bal
173     return out
174
175
176     def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
177         """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.
178
179         Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
180         and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
181         reconciles, so the caller can refuse to persist.
182         """
183         with pdfplumber.open(pdf_path, password=password) as pdf:
184             words_by_page = [
185                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
186             ]
187             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
188             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
189
190             account_number = profile.first_account(head) or "UNKNOWN"
191             summary = parse_statement_summary(head)
192             oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)
193
194             confirmed: list[ConceptRow] = []
195             for table in tables:
196                 res = map_table(table, profile, oracle=oracle)
197                 if res and res.verified:
198                     confirmed.extend(res.rows)
199             if confirmed and oracle(confirmed):
200                 return SavingsParse(account_number, confirmed, True)
201
202             matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
203             clustered = map_table(matrix, profile, oracle=oracle)
204             if clustered and clustered.verified:
205                 return SavingsParse(account_number, clustered.rows, True)
206
207             # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
208             #     debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
209             #     direction from the running balance and let the oracle cross-check amount ==
|?balance|.
210             delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
211             if delta and oracle(delta):
212                 return SavingsParse(account_number, delta, True)

```

```

213
214         fallback = confirmed or (clustered.rows if clustered else [])
215         return SavingsParse(account_number, fallback, False)
216
217
218     __all__ = [
219         "SavingsParse",
220         "parse_savings",
221         "parse_statement_summary",
222         "txn_line_predicate",
223         "reconcile_oracle",
224     ]
225

```

3. user (2026-06-10T16:27:48.184Z)

```

src\muneem\parsers\__init__.py
src\muneem\parsers\concepts.py
src\muneem\parsers\validation.py
src\muneem\parsers\clustering.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\engine.py
src\muneem\parsers\base.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\hdfc.py

```

4. user (2026-06-10T16:27:54.487Z)

```

1     """The verifier ? reconciliation as a layout-independent truth oracle.
2
3     Reconciliation proves a parse is complete and correct without inspecting any sensitive
value**:
4     if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5     parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6     money ? extraction proposes, reconciliation disposes (see
7     [docs/parser-architecture.md](../docs/parser-architecture.md) §2, §5).
8
9     This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11     * per-row walk ? reconcile_running_balance: balance[i] == balance[i-1] ? debit
+ credit.
12     * total reconcile ? reconcile_total: opening + ?credit ? ?debit == closing
(work even
13     with no per-row balance, e.g. Axis savings).
14     * bookend ? verify: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15     * selection ? select_reconciling: among candidate concept mappings, pick the one
arithmetic
16     endorses (the seam the L2 mapping search plugs into).
17     * confidence ? Confidence + BookendReport.confidence: a recorded verdict,
not an
18     assumption.
19
20     Reports carry counts / booleans / a verdict only ? never an amount.
21     """
22
23     from __future__ import annotations
24
25     import re
26     from collections.abc import Sequence

```

```

27 from dataclasses import dataclass
28 from enum import StrEnum
29
30 from .concepts import ConceptRow, StatementConcepts
31
32 _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34
35 def parse_amount(value: object) -> float | None:
36     """Parse a statement money cell to a float.
37
38     Returns ``None`` only for a genuinely absent cell (None / empty / ``"-`` /
39     non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
40     *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
41     ``None`` and would corrupt a real zero balance and the reconciliation math.
42     """
43     if value is None:
44         return None
45     text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
46     if text in ("", "-"):
47         return None
48     try:
49         return float(text)
50     except ValueError:
51         return None
52
53
54 @dataclass
55 class ReconcileReport:
56     """Result of a running-balance walk. Counts only, never an amount."""
57
58     n: int # rows considered
59     compared: int # consecutive (balance-bearing) pairs actually checked
60     breaks: int
61     first_break: int | None = None
62
63     @property
64     def ok(self) -> bool:
65         return self.compared > 0 and self.breaks == 0
66
67
68 def reconcile_running_balance(
69     entries: Sequence[tuple[float | None, float | None, float | None]],
70     tol: float = 0.01,
71 ) -> ReconcileReport:
72     """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
73
74     ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
75     ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
76     is skipped (can't anchor on it). Because the walk spans the whole statement, a
77     clean result also rules out any row missed *between* two balances ? which is the
78     failure mode silent ``extract_tables()`` row-drops would otherwise hide.
79     """
80     breaks = 0
81     first_break: int | None = None
82     compared = 0
83     prev: float | None = None
84     for i, (debit, credit, balance) in enumerate(entries):
85         if balance is None:
86             continue
87         if prev is not None:
88             expected = prev - (debit or 0.0) + (credit or 0.0)
89             compared += 1
90             if abs(expected - balance) > tol:
91                 breaks += 1

```

```

92         if first_break is None:
93             first_break = i
94         prev = balance
95         return ReconcileReport(
96             n=len(entries), compared=compared, breaks=breaks, first_break=first_break
97         )
98
99
100     def _row_entries(
101         rows: Sequence[ConceptRow],
102     ) -> list[tuple[float | None, float | None, float | None]]:
103         """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
104         consumes."""
105         return [(r.debit, r.credit, r.balance) for r in rows]
106
107     def reconcile_total(
108         rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
109     ) -> bool:
110         """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.
111
112         This is the oracle for statements with no reliable per-row balance (e.g. Axis
113         savings):
114         it checks the whole statement against its own opening/closing bookends without
115         needing each
116         row's running balance. ``opening`` and ``closing`` come from the bank's
117         SUMMARY/header
118         (``StatementConcepts``); the caller must have them.
119         """
120         delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
121         return abs((opening + delta) - closing) <= tol
122
123     class Confidence(StrEnum):
124         """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
125         values).
126
127         ``RECONCILED`` ? the parse passed every available check, so it is trusted.
128         ``UNRECONCILED`` ?
129         a check failed (or none could run); the parse is suspect and is quarantined rather
130         than trusted.
131         """
132         RECONCILED = "reconciled"
133         UNRECONCILED = "unreconciled"
134
135     @dataclass
136     class BookendReport:
137         """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
138
139         ``None`` for a check means the figure wasn't available to verify (not a failure).
140         ``ok``
141         requires the per-row balance walk plus every available check to pass.
142         """
143         n: int
144         reconciles: bool
145         debit_count_ok: bool | None = None
146         credit_count_ok: bool | None = None
147         opening_ok: bool | None = None
148         closing_ok: bool | None = None
149
150         @property
151         def ok(self) -> bool:

```



```

149         checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
150         return self.n > 0 and self.reconciles and all(c is True for c in checks)
151
152     @property
153     def confidence(self) -> Confidence:
154         return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED
155
156
157     def verify(
158         rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
159     ) -> BookendReport:
160         """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.
161
162         The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
163         extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
164         figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
165         implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
166         """
167         reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
168         debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
169         if concepts.debit_count is not None:
170             debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
171         if concepts.credit_count is not None:
172             credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
173         if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
174             closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
175         if rows and rows[0].balance is not None and concepts.opening_balance is not None:
176             implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
177             opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
178         return BookendReport(
179             n=len(rows),
180             reconciles=reconciles,
181             debit_count_ok=debit_count_ok,
182             credit_count_ok=credit_count_ok,
183             opening_ok=opening_ok,
184             closing_ok=closing_ok,
185         )
186
187
188     def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
189         """Among candidate concept-row mappings, return the index of the first that
reconciles.
190
191         This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.

```

```

198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215

```

5. user (2026-06-10T16:27:55.021Z)

```

400         )
401         return out
402
403
404     @register_parser
405     class HDFCParser(StatementParser):
406         """Registered parser for HDFC savings / transaction-account statements."""
407
408         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409             low = (text or "").lower()
410             is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"").lower()
411             is_statement = "savings account details" in low or (
412                 "narration" in low and "closing balance" in low
413             )
414             return is_hdfc and is_statement
415
416         def parse_and_insert(
417             self,
418             pdf_path: Path,
419             conn,
420             doc_id: int,
421             password: str | None = None,
422         ) -> int:
423             """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425             Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426             balance walk (and the bank's SUMMARY when present) so a downstream consumer
427             can tell a trustworthy parse from a suspect one. Returns the row count.
428             """
429             try:
430                 stmt = parse_statement(pdf_path, password=password or "")
431             except ParseError:
432                 raise
433             except Exception as exc:
434                 raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
435             for t in stmt.transactions:
436                 dbmod.insert_hdfc_transaction(
437                     conn,
438                     document_id=doc_id,
439                     account_number=stmt.account_number,
440                     txn_date=t.date,

```

```

441         value_date=t.value_date,
442         particulars=t.narration,
443         ref=t.ref,
444         withdrawal=t.withdrawal,
445         deposit=t.deposit,
446         closing_balance=t.closing_balance,
447     )
448     summary = parse_summary(pdf_path, password=password or "")
449     ok = (
450         validate_against_summary(stmt, summary).ok
451         if summary is not None
452         else reconcile(stmt).ok
453     )
454     verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455     dbmod.record_parse(
456         conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457         parser_version=__version__
458     )
459     return len(stmt.transactions)

```

6. user (2026-06-10T16:28:13.172Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
4     table +
5     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
6     gated** so a
7     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
8     Axis specifics
9     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
10    mapping.
11
12    Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
13    reconcile
14    (parse/persist split + count only). No statement content is logged here.
15    """
16
17    from __future__ import annotations
18
19    import re
20    import sqlite3
21    from dataclasses import dataclass
22    from pathlib import Path
23
24    import pdfplumber
25
26    from muneem import __version__
27    from muneem import db as dbmod
28    from muneem.errors import ParseError
29    from muneem.redaction import redact_filename
30
31    from . import register_parser
32    from .base import StatementParser
33    from .profiles import AXIS_PROFILE
34    from .savings import SavingsParse, parse_savings, parse_statement_summary,
35    txn_line_predicate
36    from .validation import Confidence, parse_amount
37
38    _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
39    _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})", re.compile(r"(\d{6}\*\d{4})")]
40
41    def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:

```

```

37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(AxisPROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, AxisPROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66     def parse_and_insert(
67         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68     ) -> int:
69         """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72         parser can't yet read surfaces loudly instead of saving mislabeled rows.
73         """
74         try:
75             parsed = parse_axis_savings(pdf_path, password)
76         except ParseError:
77             raise
78         except Exception as exc:
79             # Filename only ? never statement text; original cause kept via `from`.
80             name = redact_filename(pdf_path)
81             raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83         if not parsed.verified:
84             dbmod.record_parse(
85                 conn,
86                 doc_id,
87                 confidence=Confidence.UNRECONCILED.value,
88                 parser="AxisParser",
89                 parser_version=__version__,
90             )
91             return 0
92
93         for r in parsed.rows:
94             dbmod.insert_axis_transaction(

```

```

95         conn,
96         document_id=doc_id,
97         account_number=parsed.account_number,
98         txn_date=r.txn_date,
99         value_date=r.value_date or "",
100        particulars=r.narration,
101        debit=r.debit,
102        credit=r.credit,
103        balance=r.balance,
104    )
105    dbmod.record_parse(
106        conn,
107        doc_id,
108        confidence=Confidence.RECONCILED.value,
109        parser="AxisParser",
110        parser_version=__version__,
111    )
112    return len(parsed.rows)
113
114
115    # --- Credit card (no running balance; count only)
116    -----
117
118    @dataclass
119    class AxisCCTxn:
120        txn_date: str
121        particulars: str
122        merchant_category: str
123        amount: float
124        type: str # "Cr" or "Dr"
125
126
127    def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128        """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129        rows: list[AxisCCTxn] = []
130        for table in tables or []:
131            for row in table or []:
132                if not row or len(row) < 9:
133                    continue
134                date_str = str(row[0]).strip() if row[0] else ""
135                if not _CC_DATE_RE.match(date_str):
136                    continue
137                amount_str = str(row[8]).strip() if row[8] else ""
138                is_cr = "Cr" in amount_str
139                amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                rows.append(
141                    AxisCCTxn(
142                        txn_date=date_str,
143                        particulars=str(row[2]).strip() if row[2] else "",
144                        merchant_category=str(row[7]).strip() if row[7] else "",
145                        amount=amount,
146                        type="Cr" if is_cr else "Dr",
147                    )
148                )
149        return rows
150
151
152    def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153        """Open an Axis credit-card statement and return (card_number, transactions)."""
154        card_number = "UNKNOWN"
155        rows: list[AxisCCTxn] = []
156        with pdfplumber.open(pdf_path, password=password) as pdf:
157            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,

```

```

len(pdf.pages)))
158         card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159         for page in pdf.pages:
160             rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192
193             # record provenance (with no confidence verdict).
194             dbmod.record_provenance(
195                 conn,
196                 document_id=doc_id,
197                 source_type="pdf_statement",
198                 parser="AxisCreditCardParser",
199                 parser_version=__version__,
200                 mapping_layer="deterministic",
201             )
202             return len(rows)

```

7. user (2026-06-10T16:28:13.472Z)

```

1     """AU Small Finance Bank savings / transaction-account statement parser.
2
3     Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
5     gated** so a
6     vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
7     entirely in
8     ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
9     here.
10    """
11
12    from __future__ import annotations

```

```

11     import sqlite3
12     from pathlib import Path
13
14     from muneem import __version__
15     from muneem import db as dbmod
16     from muneem.errors import ParseError
17     from muneem.redaction import redact_filename
18
19     from . import register_parser
20     from .base import StatementParser
21     from .profiles import AUBANK_PROFILE
22     from .savings import SavingsParse, parse_savings
23     from .validation import Confidence
24
25
26     def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
27         """Parse an AU Bank savings statement via the shared engine-driven path."""
28         return parse_savings(pdf_path, AUBANK_PROFILE, password)
29
30
31     @register_parser
32     class AUParser(StatementParser):
33         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
34             if sender and "aubank" in sender.lower():
35                 return True
36             low = (text or "").lower()
37             return "au small finance bank" in low or "aubank" in low
38
39         def parse_and_insert(
40             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
41         ) -> int:
42             """Persist a savings parse **only if it reconciles**; otherwise store
nothing."""
43             try:
44                 parsed = parse_au_savings(pdf_path, password)
45             except ParseError:
46                 raise
47             except Exception as exc:
48                 raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc
49
50             if not parsed.verified:
51                 dbmod.record_parse(
52                     conn,
53                     doc_id,
54                     confidence=Confidence.UNRECONCILED.value,
55                     parser="AUParser",
56                     parser_version=__version__,
57                 )
58             return 0
59
60         for r in parsed.rows:
61             dbmod.insert_au_transaction(
62                 conn,
63                 document_id=doc_id,
64                 account_number=parsed.account_number,
65                 txn_date=r.txn_date,
66                 value_date=r.value_date or "",
67                 particulars=r.narration,
68                 debit=r.debit,
69                 credit=r.credit,
70                 balance=r.balance,
71             )

```

```

72         dbmod.record_parse(
73             conn,
74             doc_id,
75             confidence=Confidence.RECONCILED.value,
76             parser="AUParser",
77             parser_version=__version__,
78         )
79     return len(parsed.rows)
80

```

8. user (2026-06-10T16:28:50.201Z)

```

1     """HDFC savings / transaction-account statement parser.
2
3     **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4     default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5     table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6     decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7     position. pdfplumber's whole chain is permissive and it opens password-protected
8     statements directly.
9
10    **Layout.** Each transaction is one *primary* line ? the transaction date at the
11    far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12    Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13    narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14    cheque/reference number. We group words into visual lines by ``top``, detect
15    primary lines by a leading date + a closing-balance amount, attach continuations,
16    and read columns by x-position.
17
18    **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19    balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20    ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21    statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22    parse complete without inspecting any individual value ? which is how we validate
23    against real (sensitive) statements. No statement content is logged here.
24    """
25
26    from __future__ import annotations
27
28    import csv
29    import re
30    from dataclasses import dataclass, field
31    from pathlib import Path
32
33    import pdfplumber
34
35    from muneem import __version__
36    from muneem import db as dbmod
37    from muneem.errors import ParseError
38    from muneem.redaction import redact_filename
39
40    from . import register_parser
41    from .base import StatementParser
42    from .concepts import ConceptRow, StatementConcepts
43    from .profiles import HDFC_PROFILE
44    from .validation import (
45        BookendReport,
46        Confidence,
47        ReconcileReport,
48        reconcile_running_balance,
49        verify,
50    )
51
52    # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
53    # Left-aligned columns key on x0; right-aligned amount columns key on their right

```



```

54 # edge (x1), which is stable regardless of the number's magnitude.
55 _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
56 _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
57 _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
58 _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
59 _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
60 _LINE_TOL = 2.5 # words within this many points of each other share a visual line
61 _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
62
63 _DATE_RE = re.compile(r"^\d{2}/\d{2}/\d{4}$")
64 # Indian-format money: optional minus, comma groups, exactly two decimals, optional
Cr/Dr.
65 _AMOUNT_RE = re.compile(r"^-?\d[,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
66 _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
67
68
69 @dataclass
70 class Transaction:
71     date: str # dd/mm/yyyy, as printed
72     narration: str
73     withdrawal: float
74     deposit: float
75     closing_balance: float
76     value_date: str | None = None
77     ref: str | None = None
78     page: int = 0
79
80     def to_concept(self) -> ConceptRow:
81         """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
verifier's input).
82
83         Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
``page``
84         field is HDFC-internal provenance and has no concept equivalent.
85         """
86         return ConceptRow(
87             txn_date=self.date,
88             narration=self.narration,
89             debit=self.withdrawal,
90             credit=self.deposit,
91             balance=self.closing_balance,
92             value_date=self.value_date,
93             reference=self.ref,
94         )
95
96
97 @dataclass
98 class Statement:
99     path: Path
100     transactions: list[Transaction] = field(default_factory=list)
101     account_number: str | None = None
102
103
104 # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
vocabulary +
105 # verifier (the concept-core refactor). These back-compat names keep HDFC's public
surface stable:
106 # StatementSummary == the statement-level concepts a SUMMARY block supplies
107 # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
checks)
108 StatementSummary = StatementConcepts
109 ValidationReport = BookendReport
110
111
112 def _to_amount(token: str) -> float | None:

```

```

113     """Parse an HDFC money token to a float, or None if it is not money.
114
115     Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
116     is real information and the reconciliation math needs it.
117     """
118     if not _AMOUNT_RE.match(token):
119         return None
120     cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "", "")
121     try:
122         return float(cleaned)
123     except ValueError:
124         return None
125
126
127 def _group_lines(words: list[dict]) -> list[list[dict]]:
128     """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
129     lines: list[list[dict]] = []
130     anchor_top: float | None = None
131     for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
132         if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
133             lines[-1].append(w)
134         else:
135             lines.append([w])
136             anchor_top = w["top"]
137     return [sorted(line, key=lambda w: w["x0"]) for line in lines]
138
139
140 def _line_top(line: list[dict]) -> float:
141     return min(w["top"] for w in line)
142
143
144 def _has_narration(line: list[dict]) -> bool:
145     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
146
147
148 def _has_amount(line: list[dict]) -> bool:
149     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
150
151
152 def _has_leading_date(line: list[dict]) -> bool:
153     first = line[0]
154     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
155
156
157 def _has_footer_marker(line: list[dict]) -> bool:
158     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
159
160
161 def _is_primary(line: list[dict]) -> bool:
162     """A transaction's first line: a date at the far left + a closing-balance amount.
163
164     Requiring the closing-balance amount guards against stray dates in the page
165     header / account-summary box (e.g. the statement period).
166     """
167     if not _has_leading_date(line):
168         return False
169     return any(
170         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
171         _AMOUNT_RE.match(w["text"])
172         for w in line
173     )
174
175 def _is_continuation(line: list[dict]) -> bool:
176     """A narration-wrap line: narration-band text, no leading date, no amounts."""

```

```

177         return _has_narration(line) and not _has_leading_date(line) and not
_has_amount(line)
178
179
180     def _parse_transaction(
181         primary: list[dict], continuations: list[list[dict]], page_no: int
182     ) -> Transaction:
183         withdrawal = deposit = 0.0
184         closing: float | None = None
185         narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
186
187         for line in (primary, *continuations):
188             for w in line:
189                 x0, x1, text = w["x0"], w["x1"], w["text"]
190                 if x0 >= _AMOUNT_MIN_X0:
191                     val = _to_amount(text)
192                     if val is None:
193                         continue
194                     if x1 < _WITHDRAWAL_MAX_X1:
195                         withdrawal = val
196                     elif x1 < _DEPOSIT_MAX_X1:
197                         deposit = val
198                     else:
199                         closing = val
200                 elif x0 >= _NARR_MIN_X0:
201                     narr.append((w["top"], x0, text))
202
203         value_date: str | None = None
204         parts: list[str] = []
205         for _top, _x0, text in sorted(narr):
206             if value_date is None and _DATE_RE.match(text):
207                 value_date = text # the value date, lifted out of the narration band
208                 continue
209             parts.append(text)
210
211         ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
212         ref = max(ref_candidates, key=len) if ref_candidates else None
213
214         return Transaction(
215             date=primary[0]["text"],
216             narration=" ".join(parts),
217             withdrawal=withdrawal,
218             deposit=deposit,
219             closing_balance=closing if closing is not None else 0.0,
220             value_date=value_date,
221             ref=ref,
222             page=page_no,
223         )
224
225
226     def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
227         """Pure, PDF-free core: turn grouped table-body lines into transactions.
228
229         Kept separate from PDF I/O so the column logic is unit-testable on synthetic
230         word lists, with no real data.
231         """
232         primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
233         transactions: list[Transaction] = []
234         for k, idx in enumerate(primaries):
235             stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
236             primary = lines[idx]
237             primary_top = _line_top(primary)
238             continuations: list[list[dict]] = []
239             for m in range(idx + 1, stop):

```

```

240         line = lines[m]
241         if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
242             break
243             if _is_continuation(line):
244                 continuations.append(line)
245                 transactions.append(_parse_transaction(primary, continuations, page_no))
246         return transactions
247
248
249 def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
250     """Lines below the column header (drops the per-page account-summary box)."""
251     header_top: float | None = None
252     for line in lines:
253         texts = {w["text"] for w in line}
254         if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255             header_top = _line_top(line)
256             break
257     if header_top is None:
258         return []
259     return [line for line in lines if _line_top(line) > header_top + 1]
260
261
262 def _extract_account_number(text: str) -> str | None:
263     """Account number via the HDFC profile (bank knowledge as data; same pattern as
before)."""
264     return HDFC_PROFILE.first_account(text)
265
266
267 def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
268     """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
269
270     ``password`` is forwarded to pdfplumber for encrypted statements and used
271     in-memory only ? never stored or logged.
272     """
273     path = Path(pdf_path)
274     transactions: list[Transaction] = []
275     account_number: str | None = None
276     with pdfplumber.open(str(path), password=password or "") as pdf:
277         for page_no, page in enumerate(pdf.pages):
278             if account_number is None and page_no < 2:
279                 account_number = _extract_account_number(page.extract_text() or "")
280                 words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
281                 body = _table_body(_group_lines(words))
282                 transactions.extend(extract_transactions_from_lines(body, page_no))
283     return Statement(path=path, transactions=transactions,
account_number=account_number)
284
285
286 def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
287     """Check the running balance via the shared reconciler (closing[i] ==
288     closing[i-1] - withdrawal + deposit).
289
290     Spans page breaks (transactions are in document order), so a clean report also
291     validates page stitching. Reports counts only ? no amounts.
292     """
293     entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
statement.transactions]
294     return reconcile_running_balance(entries, tol)
295
296
297 def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
298     """Pure core: read HDFC's SUMMARY block from extracted text lines.
299
300     Layout (stable across statements)::

```

```

301
302     SUMMARY
303     Opening Balance  Debit Amount  Credit Amount  Closing Balance
304     <opening>      <debitAmt>    <creditAmt>    <closing>
305     Debit Count    Credit Count
306     <debitCount>  <creditCount>
307
308     The values land on their own lines, so we take the first post-SUMMARY line
309     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
310     integers and no money (the counts).
311     """
312     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
313     if start is None:
314         return None
315     balances: list[float] | None = None
316     counts: list[int] | None = None
317     for ln in lines[start + 1 : start + 8]:
318         tokens = ln.split()
319         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
320         ints = [int(t) for t in tokens if t.isdigit()]
321         if balances is None and len(monies) >= 4:
322             balances = monies[:4]
323         elif counts is None and not monies and len(ints) >= 2:
324             counts = ints[:2]
325     if balances is None and counts is None:
326         return None
327     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328     opening, debit_amt, credit_amt, closing = bvals[:4]
329     cvals: list[int | None] = [*counts] if counts else [None, None]
330     debit_count, credit_count = cvals[:2]
331     return StatementSummary(
332         opening_balance=opening,
333         closing_balance=closing,
334         debit_count=debit_count,
335         credit_count=credit_count,
336         debit_amount=debit_amt,
337         credit_amount=credit_amt,
338     )
339
340
341 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344         for page in pdf.pages:
345             text = page.extract_text() or ""
346             if "SUMMARY" in text:
347                 summary = _parse_summary_lines(text.splitlines())
348                 if summary is not None:
349                     return summary
350     return None
351
352
353 def validate_against_summary(
354     statement: Statement, summary: StatementSummary, tol: float = 0.01
355 ) -> ValidationReport:
356     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357
358     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
361     closing
362     balances catch a missing first/last row.
363     """
364     rows = [t.to_concept() for t in statement.transactions]
365     return verify(rows, summary, tol)

```

```

365
366
367 def write_csv(statement: Statement, out_path: Path | str) -> Path:
368     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
369
370     Writes real data ? call only for a local, gitignored destination.
371     """
372     out = Path(out_path)
373     out.parent.mkdir(parents=True, exist_ok=True)
374     with open(out, "w", newline="", encoding="utf-8") as fh:
375         writer = csv.writer(fh)
376         writer.writerow(
377             [
378                 "date",
379                 "value_date",
380                 "narration",
381                 "ref",
382                 "withdrawal",
383                 "deposit",
384                 "closing_balance",
385                 "page",
386             ]
387         )
388         for t in statement.transactions:
389             writer.writerow(
390                 [
391                     t.date,
392                     t.value_date or "",
393                     t.narration,
394                     t.ref or "",
395                     f"{t.withdrawal:.2f}",
396                     f"{t.deposit:.2f}",
397                     f"{t.closing_balance:.2f}",
398                     t.page,
399                 ]
400             )

```

9. user (2026-06-10T16:28:50.240Z)

```

1     """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3     This is the *generic-first* half of the concept-core architecture (see
4     [docs/parser-architecture.md](../docs/parser-architecture.md) §4, §6).
5     Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
6     or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7     via a cheap-first ladder, with a profile supplying priors:
8
9     * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
10    * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
11    * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
12    plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
13    reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
14    makes the math work* rather than by its label.
15
16    The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
17    names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
18    (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see

```

```

19     the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20     value; provenance is the winning layer + column map.
21     """
22
23     from __future__ import annotations
24
25     import itertools
26     import re
27     from collections.abc import Callable, Sequence
28     from dataclasses import dataclass, field
29
30     from .concepts import ConceptRow
31     from .validation import parse_amount
32
33     Cell = object # a table cell: str | None (we coerce defensively)
34     Row = Sequence[Cell]
35
36     # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
37     _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
38         re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy
39         re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy
40     )
41     # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
42     # for an amount column during content classification.
43     _MONEY_RE = re.compile(r"(?i)^\d{1,2}(?:\s*(?:cr|dr))?$")
44     _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
45
46     # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
47     # lower-cased header labels matched exactly (not by substring) during L0.
48     _BANK_ALIASES: dict[str, tuple[str, ...]] = {
49         "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
50         "value_date": ("value date", "val date", "value dt"),
51         "narration": (
52             "narration",
53             "particulars",
54             "description",
55             "transaction details",
56             "details",
57             "remarks",
58             "transaction",
59             "transaction remarks",
60         ),
61         "reference": (
62             "ref",
63             "reference",
64             "ref no",
65             "reference no",
66             "cheque no",
67             "chq no",
68             "instrument no",
69         ),
70         "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
71         "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
72         "balance": (
73             "balance",
74             "closing balance",
75             "running balance",
76             "closing bal",
77             "available balance",
78         ),
79     }

```

```

80
81
82     def _text(cell: Cell) -> str:
83         return "" if cell is None else str(cell).strip()
84
85
86     def _norm(label: str) -> str:
87         return re.sub(r"\s+", " ", label.strip().lower())
88
89
90     def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
91         return any(p.match(text) for p in patterns)
92
93
94     def _looks_like_money(text: str) -> bool:
95         return bool(_MONEY_RE.match(text))
96
97
98     @dataclass
99     class FamilySpec:
100         """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
101
102         The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
103         future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
104         ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105         """
106
107         name: str
108         concepts: tuple[str, ...]
109         date_concepts: tuple[str, ...] # in order: the first is the primary (row-gating)
date
110         amount_concepts: tuple[str, ...] # money columns; the non-balance ones are the
"flows"
111         balance_concept: str | None # the running-balance concept, if the family has one
112         text_concepts: tuple[str, ...] # free-text columns; the first gets the widest text
column
113         default_aliases: dict[str, tuple[str, ...]]
114         build: Callable[[list[dict[str, str]]], list]
115
116
117     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
118         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134
135     BANK_FAMILY = FamilySpec(
136         name="bank",

```



```

137     concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
138     date_concepts=("txn_date", "value_date"),
139     amount_concepts=("debit", "credit", "balance"),
140     balance_concept="balance",
141     text_concepts=("narration", "reference"),
142     default_aliases=_BANK_ALIASES,
143     build=_build_bank_rows,
144 )
145
146
147 @dataclass
148 class MappingResult:
149     """The outcome of mapping one table: typed rows + how we got there (provenance)."""
150
151     rows: list # typed rows from family.build (e.g. list[ConceptRow])
152     mapped: list[
153         dict[str, str]
154     ] # per-row {concept: raw cell}, incl. any profile-declared extra columns
155     column_map: dict[str, int] # the winning concept -> column-index assignment
156     layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
157     verified: bool # did the oracle endorse it? (False if no oracle was supplied)
158
159
160 @dataclass
161 class Profile:
162     """Declarative per-issuer knowledge ? *data, not code* (architecture §6).
163
164     Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
165     accepted date shape, a positional fallback when there's no header, table anchors,
account-number
166     patterns, quirks, and (placeholder, wiring parked) the password rule.
167     """
168
169     name: str
170     aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
171     date_patterns: tuple[re.Pattern, ...] = ()
172     column_order: tuple[str, ...] | None = None
173     table_anchors: tuple[str, ...] = ()
174     account_patterns: tuple[re.Pattern, ...] = ()
175     quirks: frozenset[str] = frozenset()
176     password_rule: str | None = None # data only; sourcing is parked (DESIGN §8.2)
177
178     def first_account(self, text: str) -> str | None:
179         for rx in self.account_patterns:
180             m = rx.search(text)
181             if m:
182                 return m.group(1).strip()
183         return None
184
185
186     def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
187         """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
188         index: dict[str, str] = {}
189         for concept, labels in family.default_aliases.items():
190             for label in labels:
191                 index[_norm(label)] = concept
192         for concept, labels in profile.aliases.items():
193             for label in labels:
194                 index[_norm(label)] = concept
195         return index
196
197

```

```

198 def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
199     return profile.date_patterns or _GENERIC_DATE_PATTERNS
200
201
202 def _detect_header(
203     rows: list[list[Cell]], alias_index: dict[str, str]
204 ) -> tuple[int | None, dict[str, int]]:
205     """Find the header row (most exact alias hits, >=2) and its concept->column map."""
206     best_idx: int | None = None
207     best_map: dict[str, int] = {}
208     for i, row in enumerate(rows):
209         cmap: dict[str, int] = {}
210         for col, cell in enumerate(row):
211             concept = alias_index.get(_norm(_text(cell)))
212             if concept and concept not in cmap:
213                 cmap[concept] = col
214             if len(cmap) > len(best_map):
215                 best_map, best_idx = cmap, i
216     if len(best_map) < 2:
217         return None, {}
218     return best_idx, best_map
219
220
221 def _is_balance_marker(row: Row) -> bool:
222     return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)
223
224
225 def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
226     if date_col is not None:
227         cell = row[date_col] if date_col < len(row) else None
228         return _matches_date(_text(cell), patterns)
229     return any(_matches_date(_text(c), patterns) for c in row)
230
231
232 def _classify_columns(
233     data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
234 ) -> dict[int, str]:
235     """Tag each column date / amount / text / empty by majority content."""
236     kinds: dict[int, str] = {}
237     for c in range(ncols):
238         cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
239         if not cells:
240             kinds[c] = "empty"
241             continue
242         thresh = max(1, int(len(cells) * 0.6))
243         if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
244             kinds[c] = "date"
245         elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
246             kinds[c] = "amount"
247         else:
248             kinds[c] = "text"
249     return kinds
250
251
252 def _base_map(
253     col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
254 ) -> dict[str, int]:
255     """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.
256
257     Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
258     goes to the family's first text concept (if any). Amount columns are left to the
search.

```

```

259     """
260     base: dict[str, int] = {}
261     date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
262     text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
263     for concept, col in zip(family.date_concepts, date_cols, strict=False):
264         base[concept] = col
265     if text_cols and family.text_concepts:
266         base[family.text_concepts[0]] = max(
267             text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
268         )
269     return base
270
271
272     def _search_candidates(
273         base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
274     ) -> list[tuple[str, dict[str, int]]]:
275         """Enumerate plausible amount-column ? concept assignments (the L2 search space).
276
277         Family-agnostic: the "flows" are the family's amount concepts minus its balance
278         concept (for
279         banks: debit, credit). We try each amount column as the balance and assign the rest
280         to the
281         flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
282         Oct).
283         """
284         amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
285         flows = [c for c in family.amount_concepts if c != family.balance_concept]
286         bal = family.balance_concept
287         cands: list[dict[str, int]] = []
288         if bal and len(amount_cols) >= len(flows) + 1:
289             for bcol in amount_cols:
290                 rest = [c for c in amount_cols if c != bcol]
291                 for assign in itertools.permutations(rest, len(flows)):
292                     cands.append(**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
293             if flows and len(amount_cols) >= len(flows):
294                 for assign in itertools.permutations(amount_cols, len(flows)):
295                     cands.append(**base, **dict(zip(flows, assign, strict=True)))
296         return [("L2", c) for c in cands[:24]]
297
298
299     def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
300         out: list[dict[str, str]] = []
301         for row in data:
302             out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
303             col_map.items()})
304         return out
305
306
307     def map_table(
308         table: Sequence[Row] | None, # Sequence (covariant) so list[list[str | None]]
309         callers fit
310         profile: Profile,
311         family: FamilySpec = BANK_FAMILY,
312         *,
313         oracle: Callable[[list], bool] | None = None,
314     ) -> MappingResult | None:
315         """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
316         candidates.
317
318         Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
319         With an
320         ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
321         the first
322         structurally-complete candidate is returned unverified (the caller reconciles

```

```

separately).
315         Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
316         was supplied but nothing reconciled.
317         """
318         if not table:
319             return None
320         rows_in = [list(r) for r in table if r is not None]
321         if not rows_in:
322             return None
323         alias_index = _alias_index(profile, family)
324         patterns = _date_patterns(profile)
325
326         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
327         start = header_idx + 1 if header_idx is not None else 0
328         date_col_hint = col_map_hdr.get("txn_date")
329         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
330             date_col_hint = profile.column_order.index("txn_date")
331
332         data = [
333             r
334             for r in rows_in[start:]
335             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
336         ]
337         if not data:
338             return None
339         ncols = max(len(r) for r in data)
340         col_kinds = _classify_columns(data, ncols, patterns)
341         base = _base_map(col_kinds, data, family)
342         flows = [c for c in family.amount_concepts if c != family.balance_concept]
343
344         candidates: list[tuple[str, dict[str, int]]] = []
345         if flows and all(c in col_map_hdr for c in flows): # L0: header pinned the flow
concepts
346             candidates.append(("L0", {**base, **col_map_hdr}))
347         elif col_map_hdr: # header present but didn't pin the flows: keep its hints, search
the rest
348             candidates.append(("L0", {**base, **col_map_hdr}))
349         if profile.column_order: # positional prior (no/partial header)
350             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351         candidates += _search_candidates(base, col_kinds, family)
352
353         primary_date = family.date_concepts[0] if family.date_concepts else None
354         best: MappingResult | None = None
355         for layer, cmap in candidates:
356             if primary_date and primary_date not in cmap:
357                 continue
358             if not (set(family.amount_concepts) & cmap.keys()):
359                 continue
360             mapped = _apply_map(data, cmap)
361             typed = family.build(mapped)
362             if oracle is None:
363                 return MappingResult(typed, mapped, cmap, layer, verified=False)
364             if oracle(typed):
365                 return MappingResult(typed, mapped, cmap, layer, verified=True)
366             if best is None:
367                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
368         return best
369
370
371 __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372

```

10. user (2026-06-10T16:29:34.926Z)

```
docs\CURRENT_STATE.md:30:[Omitted long matching line]
docs\CURRENT_STATE.md:55:* **Recover the May 2025 vintage**: the clusterer drops one transaction
line ? incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores
nothing; June 2025 was recovered via balance-delta reconstruction).
docs\parser-architecture.md:93:- **Confidence verdict**: `reconciled` (per-row or total) +
`bookended` ? trust; else quarantine.
docs\parser-architecture.md:110:- reconciliation breaks, or parsed counts ? printed summary ?
**quarantine** (store nothing or store
docs\parser-architecture.md:198:- **Confidence policy** ? what does the CLI/ledger do with a
`quarantined` statement (skip, show with
docs\ROADMAP.md:36:> **Remaining**: recover the dropped line (smarter line recovery in the
clusterer) for May 2025 and
docs\ROADMAP.md:48:2. **Axis May 2025 recovery** ? recover the one transaction line the
clusterer drops (incomplete
tests\parsers\test_validation.py:87:def
test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
tests\parsers\test_savings.py:6:(a balance jump matching no single amount) which must NOT
reconcile (the real Axis May 2025 case).
tests\parsers\test_savings.py:41: # so the reconstruction must NOT reconcile ? the oracle
quarantines an incomplete ledger (May).
src\muneeem\parsers\savings.py:75:def reconcile_oracle(opening: float | None, closing: float |
None):
src\muneeem\parsers\savings.py:192: oracle = reconcile_oracle(summary.opening_balance,
summary.closing_balance)
src\muneeem\parsers\savings.py:223: "reconcile_oracle",
src\muneeem\parsers\validation.py:125: a check failed (or none could run); the parse is
suspect and is quarantined rather than trusted.
```

11. user (2026-06-10T16:29:35.678Z)

```
tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\test_offers.py
tests\test_repo_hygiene.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_registry.py
tests\parsers\test_validation.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_local_axis.py
tests\parsers\test_au.py
tests\parsers\test_local_au.py
tests\synthpdf.py
tests\test_regression_synth.py
tests\parsers\test_savings.py
tests\test_eval_cli.py
tests\conftest.py
tests\test_cli.py
tests\test_db_crypto.py
tests\parsers\test_parse_errors.py
tests\test_errors.py
tests\test_log.py
tests\test_redaction.py
tests\test_password_rules.py
tests\test_secrets.py
tests\test_statement_unlock.py
```

```
tests\test_db_crypto_ops.py
tests\test_verifier_properties.py
tests\test_dependency_licenses.py
tests\test_precommit_hook.py
tests\test_recovery.py
tests\test_db_invariants.py
tests\test_migrations.py
tests\test_provenance.py
```

12. user (2026-06-10T16:29:44.862Z)

```
1      """Tests for the shared savings path's balance-delta reconstruction
(muneem.parsers.savings).
2
3      `_reconstruct_by_balance_delta` recovers debit/credit from the running-balance delta
when columns
4      don't split cleanly (a merged amount column, an interleaved value-date column, sparse
deposits). It
5      is exercised here on synthetic cell-matrices ? no PDF, no secrets ? including the
*dropped-row* case
6      (a balance jump matching no single amount) which must NOT reconcile (the real Axis May
2025 case).
7      """
8
9      from muneem.parsers import savings
10     from muneem.parsers.profiles import AXIS_PROFILE
11     from muneem.parsers.validation import reconcile_running_balance
12
13
14     def _matrix():
15         """June-style: date | narration | withdrawal(sparse) | deposit(sparse) | value-date
| balance.
16         Opening 10,000 ? 9,000 (?1,000) ? 14,000 (+5,000) ? 12,000 (?2,000)."""
17         return [
18             ["02-08-2025", "UPI-ABC", "1,000.00", "", "01-08-2025", "9,000.00"],
19             ["05-08-2025", "SALARY", "", "5,000.00", "04-08-2025", "14,000.00"],
20             ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "12,000.00"],
21         ]
22
23
24     def _reconciles(rows):
25         return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
26
27
28     def test_reconstructs_direction_from_balance_delta_and_reconciles():
29         rows = savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE,
opening=10000.0)
30         assert len(rows) == 3
31         assert _reconciles(rows)
32         # direction derived from the sign of ?balance; the value-date column is ignored as a
flow
33         assert rows[0].debit == 1000.0 and rows[0].credit is None
34         assert rows[1].credit == 5000.0 and rows[1].debit is None
35         assert rows[2].debit == 2000.0 and rows[2].credit is None
36         assert [r.balance for r in rows] == [9000.0, 14000.0, 12000.0]
37
38
39     def test_a_dropped_transaction_does_not_reconcile():
40         # Drop a transaction (make the last balance jump): |?balance| no longer equals the
row's amount,
41         # so the reconstruction must NOT reconcile ? the oracle quarantines an incomplete
ledger (May).
42         m = _matrix()
43         m[2] = ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "1,000.00"]
44         rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
```

```

45         assert not _reconciles(rows)
46
47
48     def test_requires_opening_balance_to_fix_first_direction():
49         assert savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE, opening=None)
50 == []
51
52     def test_ignores_a_leading_non_transaction_noise_row():
53         # A header/summary line (no leading date) must be dropped, not parsed as a
54 transaction.
55         m = [["Opening Balance", "", "", "", "", "10,000.00"], *_matrix()]
56         rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
57         assert len(rows) == 3 and _reconciles(rows)

```

13. user (2026-06-10T16:29:45.811Z)

```

60         assert reconcile_total(rows, opening=10000.0, closing=9999.5)
61         assert not reconcile_total(rows, opening=10000.0, closing=9990.0)
62
63
64     def test_reconcile_total_needs_no_per_row_balance():
65         # Credit-card-style rows (balance=None) still validate against the opening/closing
66 bookends.
67         rows = [
68             ConceptRow(txn_date="01/03/2026", narration="charge", debit=300.0),
69             ConceptRow(txn_date="02/03/2026", narration="payment", credit=100.0),
70         ]
71         assert reconcile_total(rows, opening=1000.0, closing=800.0) # 1000 ? 300 + 100
72         assert not reconcile_total(rows, opening=1000.0, closing=999.0)
73
74     # --- bookend (verify) over the concept schema
75 -----
76
77     def test_verify_full_bookend_passes_and_reports_confidence():
78         concepts = StatementConcepts(
79             opening_balance=10000.0, closing_balance=9999.5, debit_count=2, credit_count=1
80         )
81         r = verify(_concept_rows(), concepts)
82         assert r.reconciles and r.ok
83         assert r.debit_count_ok and r.credit_count_ok and r.opening_ok and r.closing_ok
84         assert r.confidence is Confidence.RECONCILED
85
86
87     def test_verify_unavailable_figures_are_not_checked_so_trust_is_withheld():
88         # No SUMMARY figures: the per-row walk still passes, but with nothing to bookend
89 against the
90         # parse is not *trusted* (ok=False). This is why HDFC falls back to reconcile() when
91 absent.
92         r = verify(_concept_rows(), StatementConcepts())
93         assert r.reconciles
94         assert r.debit_count_ok is None and r.credit_count_ok is None
95         assert r.opening_ok is None and r.closing_ok is None
96         assert not r.ok and r.confidence is Confidence.UNRECONCILED
97
98
99     def test_verify_catches_wrong_count():
100         concepts = StatementConcepts(
101             opening_balance=10000.0, closing_balance=9999.5, debit_count=1, credit_count=1
102         )
103         r = verify(_concept_rows(), concepts) # there are 2 debits, not 1
104         assert r.reconciles and r.debit_count_ok is False and not r.ok

```

```

103
104
105     # --- selection: arithmetic picks the right candidate mapping
-----
106
107
108     def test_select_reconciling_picks_the_arithmetic_winner():
109         good = _concept_rows()
110         swapped = [ # row 2's deposit mislabeled as a withdrawal ? the walk breaks
111             ConceptRow(txn_date="05/02/2026", narration="a", debit=500.0, credit=0.0,
112                 balance=9500.0),
113             ConceptRow(txn_date="08/02/2026", narration="b", debit=2000.0, credit=0.0,
114                 balance=11500.0),
115             ConceptRow(txn_date="10/02/2026", narration="c", debit=1500.5, credit=0.0,
116                 balance=9999.5),
117         ]
118         assert select_reconciling([swapped, good]) == 1
119         assert select_reconciling([swapped]) is None
120         assert select_reconciling([]) is None

```

14. user (2026-06-10T16:29:46.292Z)

```

1     # Current State
2
3     > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4     > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5     > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-
architecture.md`](./parser-architecture.md) for the parser design.
6
7     > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8     > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9     > targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10    > follow. The parser core is deliberately **family-agnostic** (see S2), so adding an
instrument
11    > family is *new data + a reconciliation rule*, not a new pipeline.
12
13    ## 1. Core Ingestion Pipeline
14    The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
15
16    * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17    * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18    * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20    Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22    ## 2. Statement Parsers (concept-core architecture)
23    Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`_autoload_parsers`), so the registry is populated in the production path (not just under
tests).
24
25    The parser core is **concept-first and verification-first** (see [`parser-
architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-
agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the

```


column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a shared **verifier** (`validation.py`) decides trust by **arithmetic** ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future **instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as **data**, not code. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).

26
27 **Active issuers** ? we have real statements + passwords, so these get validated on real data:

28
29 * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately **not** `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating**: running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `'reconciled'`/`'unreconciled'`. Refactored onto the concept core as the **reference implementation**. ? **Verified** on the user's 6 real statements ? all reconcile **and** bookend, without inspecting any value.

30 * **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == |?balance|`. Oracle-gated, so it never persists unreconciled rows. ? **6/7** real statements reconcile + persist; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).

31 * **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK_PROFILE`. ? **4/4** real statements reconcile + persist (the cleanest issuer so far).

32
33 > **Dropped**: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

34
35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3** have landed (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution** is now wired into `muneem ingest --doc-type bank_statement` ? the hybrid unlock (keychain-cached + `password_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

36
37 See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

38
39 **## 3. Database & CLI**
40 * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_`, `hdfc_transactions`) ? documented in [`schema.md`](./schema.md). Dedup on sha256. **Versioned migrations** via a hand-rolled runner (`muneem/migrations.py`): the v4 schema is the frozen baseline, forward migrations layer on (recorded in `schema_migrations` with checksums), drift is rejected ? see [`schema.md`](./schema.md) and feedbackJune2026 P6. A unified `transactions` table (plus `holdings` for instruments) with per-row provenance/confidence is the planned direction once the design decision lands (parser-architecture.md §9; feedbackJune2026 P3).

41 * **The on-disk ledger is encrypted at rest** (decision D.2) ? **SQLCipher** transparent AES-256 via `sqlcipher3`, opened through `db.connect_encrypted()`; the 256-bit key lives in the OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext `sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).

42 * **CLI**: `ingest <pdf>` (promo, or `--doc-type bank_statement` ? **auto-unlocks** encrypted PDFs, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list <bank>`, `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`,

```

`rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-at-rest.md)).
43     * Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement
passwords, **the DB encryption key**) via keyring.
44
45     ## 4. Testing & Hygiene
46     * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine
fixtures (default CI).
47     * Security guards in `test_repo_hygiene.py` + license allowlist test.
48     * **DB invariant tripwires** (`test_db_invariants.py`, default CI): a **schema lock**
(pins `SCHEMA_VERSION` + every table/column shape ? an accidental drift fails the merge; a
deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption
guards** (production ingest must write an *encrypted* ledger; a keyed open never silently
downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).
49     * **Local git hooks** (`.github/hooks/`, enable via `make hooks`): `pre-commit` blocks
staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant
tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for
server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays
private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).
50     * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed). The suite never touches the real OS keychain (autouse fixture in
`tests/conftest.py`) ? headless-CI safe.
51
52     ## 5. Next Steps
53     Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis
6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55     * **Recover the May 2025 vintage**: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
56     * **Step 4 ? L3 local-vision fallback** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
57     * **Step 5 ? unify storage** (`transactions` + `holdings`) and begin the **instrument
family** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
58     * **Regression CI for statements** (decided 2026-06, *deferred ? not yet built*): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) (§ "Planned ? Regression CI for statement parsing").
59
60     Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher;
see [encryption-at-rest.md](./encryption-at-rest.md))._
61
62     The project stays **local-first**, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
63

```

15. user (2026-06-10T16:30:10.893Z)

```

missing-leading-half endorsed: True
misread-first-debit endorsed: True
missing-trailing-row endorsed: True

```

16. user (2026-06-10T16:30:25.124Z)

```

1     """"Local-only checks for the Axis savings parser against real encrypted statements.
2
3     Skipped unless an Axis password is available (`password_rules.yaml` or
4     `MUNEEM_TEST_AXIS_PW`); never runs in CI. The savings parser is **oracle-gated**
5     (coordinate clustering + `extract_tables` ? engine mapping ? reconciliation), so this
6     asserts the safety invariant: it either stored nothing (a vintage it can't yet read) or
7     everything it stored reconciled ? never mislabeled rows. Surfaces counts / booleans /

```

```

8     parser names only ? no statement content.
9     ""
10
11     import os
12     from pathlib import Path
13
14     import pdfplumber
15     import pytest
16
17     from muneem import db as dbmod
18     from muneem.parsers import get_parser
19     from muneem.password_rules import get_password_for_bank
20
21     AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23
24     @pytest.mark.local_statements
25     @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26     def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27         pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28         if not pw:
29             pytest.skip("needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)")
30
31         with pdfplumber.open(pdf_path, password=pw) as pdf:
32             text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
33             parser = get_parser(text)
34             if parser is None:
35                 pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37             with dbmod.connect(tmp_path / "t.db") as conn:
38                 dbmod.init_db(conn)
39                 doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40                 n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41                 status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42                 stored = dbmod.list_axis_transactions(conn)
43
44             # Oracle-gated invariant: the savings parser either stores nothing (and flags the
45             # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46             if type(parser).__name__ == "AxisParser":
47                 assert (n == 0 and stored == []) or status == "reconciled", (
48                     f"{pdf_path.name}: stored {n} row(s) with status={status} ? "
49                     "stop-gap must not persist unreconciled rows"
50                 )
51

```

17. user (2026-06-10T16:30:26.356Z)

```

tests\parsers\test_axis.py:29:    assert s.opening_balance == 1000.0 and s.closing_balance ==
2500.5
tests\parsers\test_hdfc.py:115:   assert s.opening_balance == 10000.0 and s.closing_balance ==
9999.5
tests\parsers\test_hdfc.py:125:       opening_balance=10000.0, closing_balance=9999.50,
debit_count=2, credit_count=1
tests\parsers\test_hdfc.py:182:       opening_balance=10000.0, closing_balance=9999.50,
debit_count=2, credit_count=1
tests\parsers\test_savings.py:48: def test_requires_opening_balance_to_fix_first_direction():
tests\parsers\test_validation.py:8:     reconcile_total,
tests\parsers\test_validation.py:58: def test_reconcile_total_clean_and_broken():
tests\parsers\test_validation.py:60:     assert reconcile_total(rows, opening=10000.0,
closing=9999.5)
tests\parsers\test_validation.py:61:     assert not reconcile_total(rows, opening=10000.0,

```

```

closing=9990.0)
tests\parsers\test_validation.py:64: def test_reconcile_total_needs_no_per_row_balance():
tests\parsers\test_validation.py:70:     assert reconcile_total(rows, opening=1000.0,
closing=800.0) # 1000 ? 300 + 100
tests\parsers\test_validation.py:71:     assert not reconcile_total(rows, opening=1000.0,
closing=999.0)
tests\parsers\test_validation.py:79:         opening_balance=10000.0, closing_balance=9999.5,
debit_count=2, credit_count=1
tests\parsers\test_validation.py:99:         opening_balance=10000.0, closing_balance=9999.5,
debit_count=1, credit_count=1
tests\test_verifier_properties.py:14: from muneem.parsers.validation import
reconcile_running_balance, reconcile_total
tests\test_verifier_properties.py:42:     assert reconcile_total(rows, opening=opening,
closing=closing)
tests\test_verifier_properties.py:67:     assert reconcile_total(rows, opening=opening,
closing=closing)
tests\test_verifier_properties.py:68:     assert not reconcile_total(rows, opening=opening,
closing=round(closing + 1.0, 2))

```

18. user (2026-06-10T16:31:13.314Z)

Structured output provided successfully

19. assistant (2026-06-10T16:31:19.941Z)

Verification complete. The finding is real, reproduced empirically, and the high severity is justified.